Assignment 02: Hosting and Load Balancing a Web Application Using Docker Containers

Course: DS 584 - Cloud Computing, FCIT, University of the Punjab

Instructor: Junaid Abdullah

Semester: Spring 2026

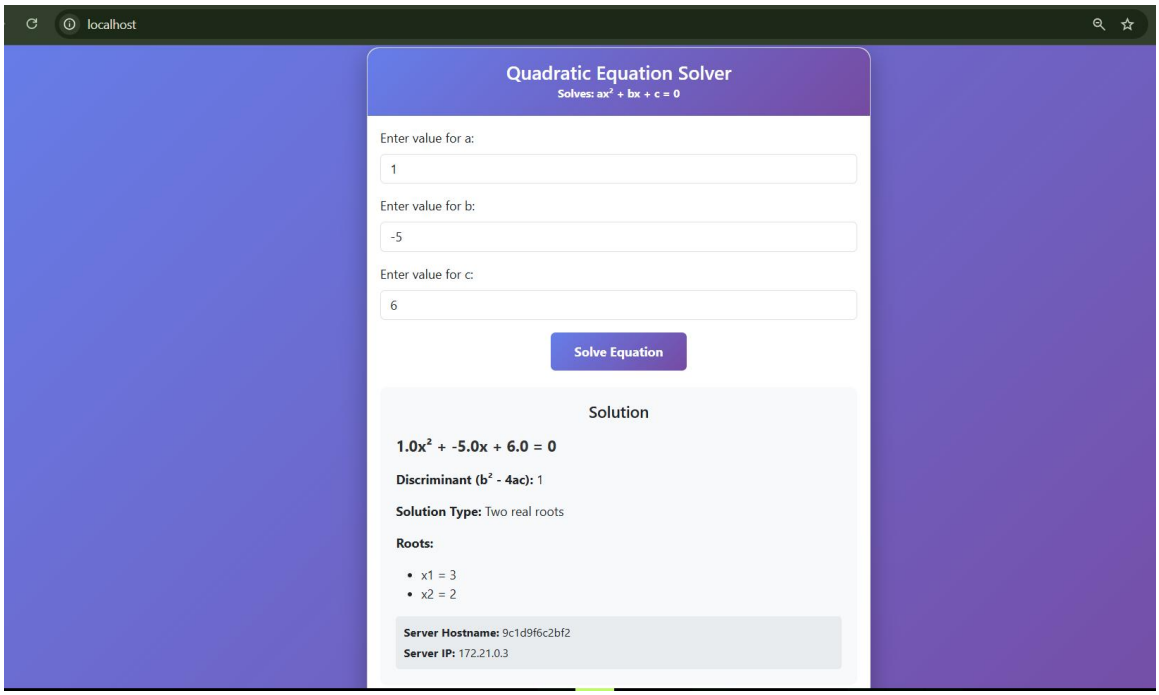
Roll #: BSDSF22A028

Name: Muhammad Shoaib Ahmad

Class: BS (DS) - Afternoon

Milestone 01: Develop a Sample Web Application

Objective: Develop a quadratic equation solver web application using HTML, JQuery, Bootstrap, and Flask (server-side). Use AJAX to submit inputs and display solution without postback.



Source Code:

app.py:

```
import math
import socket
from flask import Flask, request, jsonify, render_template

app = Flask(__name__)

def solve_quadratic(a, b, c):
    if a == 0:
        if b == 0:
            return {"roots": [], "type": "no_solution", "message": "No solution (a=0, b=0)"}
        return {"roots": [-c/b], "type": "linear", "message": f"Linear equation: x = {-c/b}"}

    discriminant = b**2 - 4*a*c

    if discriminant > 0:
        x1 = (-b + math.sqrt(discriminant)) / (2*a)
        x2 = (-b - math.sqrt(discriminant)) / (2*a)
        return {"roots": [x1, x2], "type": "two_real", "message": "Two real roots"}
```

```
    elif discriminant == 0:
        x = -b / (2*a)
        return {"roots": [x], "type": "one_real", "message": "One real root (repeated)"}
    else:
        real_part = -b / (2*a)
        imag_part = math.sqrt(abs(discriminant)) / (2*a)
        return {"roots": [f"{real_part}+{imag_part}i", f"{real_part}-{imag_part}i"], "type":
"complex", "message": "Two complex roots"}

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/solve')
def solve():
    try:
        a = float(request.args.get('a', 0))
        b = float(request.args.get('b', 0))
        c = float(request.args.get('c', 0))
    except (ValueError, TypeError):
        return jsonify({"error": "Invalid input. Please enter numeric values."}), 400

    if a == 0 and b == 0 and c == 0:
        return jsonify({"error": "All values cannot be zero"}), 400

    result = solve_quadratic(a, b, c)

    hostname = socket.gethostname()
    server_ip = socket.gethostbyname(hostname)

    return jsonify({
        "equation": f"{a}xÂ² + {b}x + {c} = 0",
        "discriminant": b**2 - 4*a*c,
        "solution": result["roots"],
        "solution_type": result["type"],
        "message": result["message"],
        "server_ip": server_ip,
        "hostname": hostname
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

templates/index.html:

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Quadratic Equation Solver</title>
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
   <style>
      body {
         background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
         min-height: 100vh;
         display: flex;
         align-items: center;
         justify-content: center;
         padding: 20px;
      }
      .card {
         border-radius: 15px;
         box-shadow: 0 10px 30px rgba(0,0,0,0.3);
      }
      .card-header {
         background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
         border-radius: 15px 15px 0 0 !important;
         color: white;
         font-weight: bold;
      }
      .btn-solve {
         background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
         border: none;
         color: white;
         padding: 12px 30px;
         font-weight: bold;
      }
      .btn-solve:hover {
         background: linear-gradient(135deg, #5568d3 0%, #6a4190 100%);
         color: white;
      }
      .result-box {
         background: #f8f9fa;
         border-radius: 10px;
         padding: 20px;
         margin-top: 20px;
```

```
            display: none;
        }
        .equation {
            font-size: 1.2em;
            font-weight: bold;
            color: #333;
        }
        .server-info {
            background: #e9ecef;
            padding: 10px;
            border-radius: 5px;
            margin-top: 15px;
            font-size: 0.9em;
        }
        .loading {
            display: none;
            text-align: center;
            margin-top: 20px;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="row justify-content-center">
            <div class="col-md-6">
                <div class="card">
                    <div class="card-header text-center py-3">
                        <h4 class="mb-0">Quadratic Equation Solver</h4>
                        <small>Solves: axÂ² + bx + c = 0</small>
                    </div>
                    <div class="card-body">
                        <form id="quadraticForm">
                            <div class="mb-3">
                                <label for="a" class="form-label">Enter value for a:</label>
                                <input type="number" class="form-control" id="a" name="a" step="any"
placeholder="Enter coefficient a" required>
                            </div>
                            <div class="mb-3">
                                <label for="b" class="form-label">Enter value for b:</label>
                                <input type="number" class="form-control" id="b" name="b" step="any"
placeholder="Enter coefficient b" required>
                            </div>
                            <div class="mb-3">
```

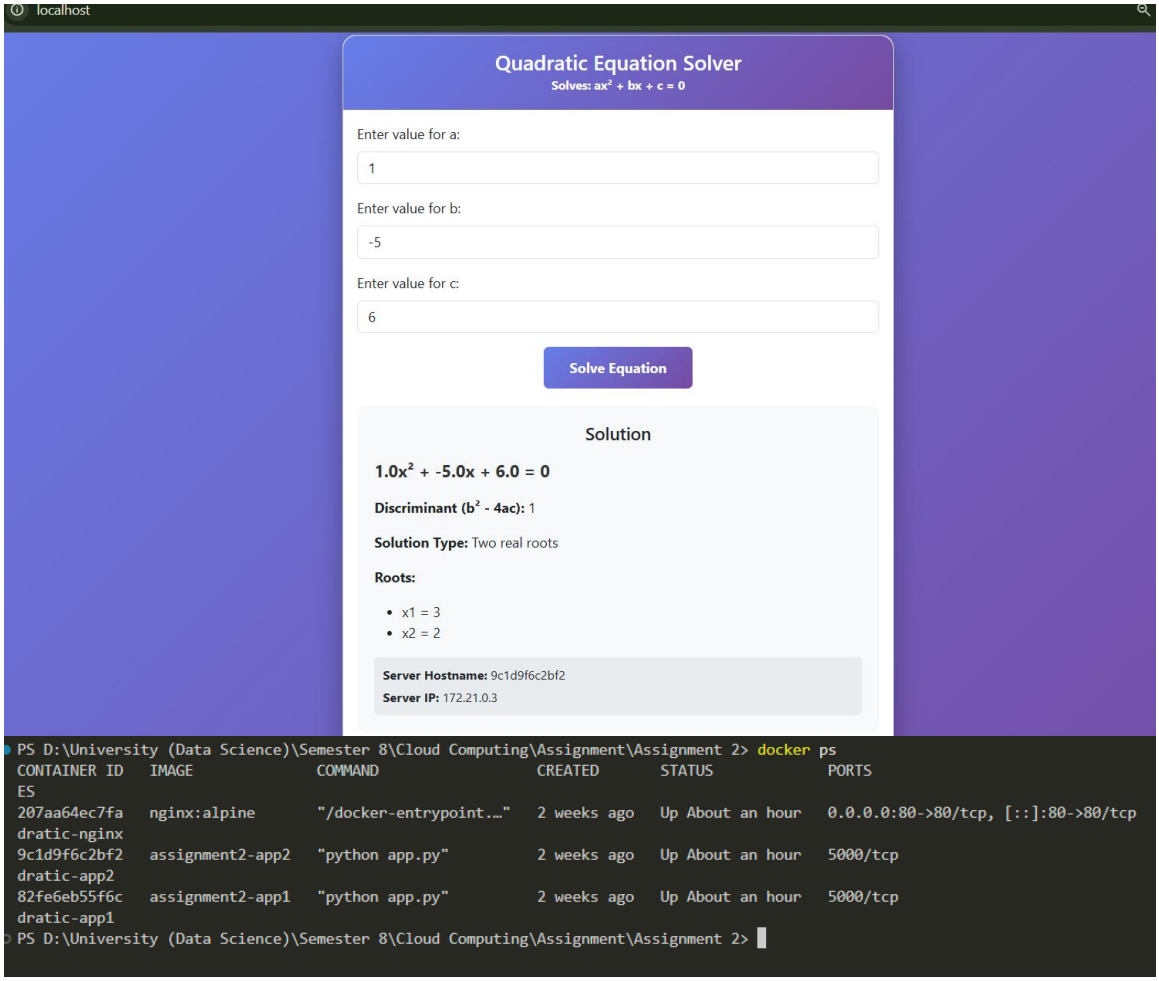
```
                <label for="c" class="form-label">Enter value for c:</label>
                <input type="number" class="form-control" id="c" name="c" step="any"
placeholder="Enter constant c" required>
              </div>
              <div class="text-center">
                <button type="submit" class="btn btn-solve">Solve Equation</button>
              </div>
            </form>

            <div class="loading" id="loading">
              <div class="spinner-border text-primary" role="status">
                <span class="visually-hidden">Loading...</span>
              </div>
              <p class="mt-2">Solving...</p>
            </div>

            <div class="result-box" id="result">
              <h5 class="text-center mb-3">Solution</h5>
              <p class="equation" id="equation"></p>
              <p><strong>Discriminant (bÂ² - 4ac):</strong> <span
id="discriminant"></span></p>
              <p><strong>Solution Type:</strong> <span
id="solutionType"></span></p>
              <p><strong>Roots:</strong></p>
              <ul id="roots"></ul>
              <div class="server-info">
                <p class="mb-1"><strong>Server Hostname:</strong> <span
id="hostname"></span></p>
                <p class="mb-0"><strong>Server IP:</strong> <span
id="serverIp"></span></p>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></sc
ript>
  <script>
```

```
    $(document).ready(function() {
      $('#quadraticForm').on('submit', function(e) {
        e.preventDefault();

        var a = $('#a').val();
        var b = $('#b').val();
        var c = $('#c').val();

        $('#loading').show();
        $('#result').hide();

        $.ajax({
          url: '/solve',
          type: 'GET',
          data: { a: a, b: b, c: c },
          dataType: 'json',
          success: function(response) {
            $('#loading').hide();
            $('#result').show();

            $('#equation').text(response.equation);
            $('#discriminant').text(response.discriminant);
            $('#solutionType').text(response.message);
            $('#hostname').text(response.hostname);
            $('#serverIp').text(response.server_ip);

            $('#roots').empty();
            if (response.solution && response.solution.length > 0) {
              $.each(response.solution, function(index, root) {
                $('#roots').append('<li>x' + (index + 1) + ' = ' + root + '</li>');
              });
            }
          },
          error: function(xhr, status, error) {
            $('#loading').hide();
            alert('Error: ' + (xhr.responseJSON && xhr.responseJSON.error ?
xhr.responseJSON.error : error));
          }
        });
      });
    });
  </script>
```

</body>
</html>

Milestone 02: Host your Web Application in a Docker Container

Objective: Prepare a Docker image of the web application and run two instances. Verify accessibility from host OS browser.



Source Code:

Dockerfile:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

docker-compose.yml:

```
services:
  app1:
    build: .
    container_name: quadratic-app1
    expose:
      - "5000"
    environment:
      - FLASK_APP=app.py
    networks:
      - quadratic-network

  app2:
    build: .
    container_name: quadratic-app2
    expose:
      - "5000"
    environment:
      - FLASK_APP=app.py
    networks:
      - quadratic-network

  nginx:
    image: nginx:alpine
    container_name: quadratic-nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app1
      - app2
    networks:
      - quadratic-network

networks:
  quadratic-network:
    driver: bridge
```

Milestone 03: Deploy Nginx as a Proxy Server in a Docker Container

Objective: Use Nginx as a reverse proxy and load balancer to distribute traffic between the two application containers.

[SCREENSHOT 4: Browser via nginx showing load balancing (changing server IPs)]

[SCREENSHOT 5: docker ps output with nginx, app1, app2 running]

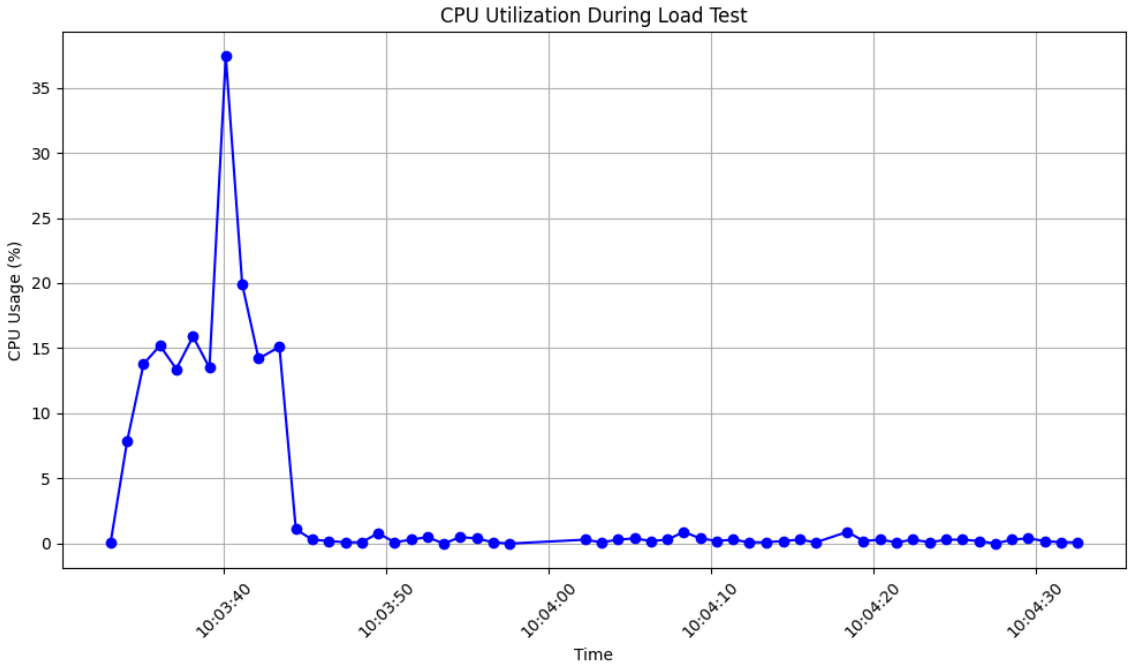
Nginx Configuration:

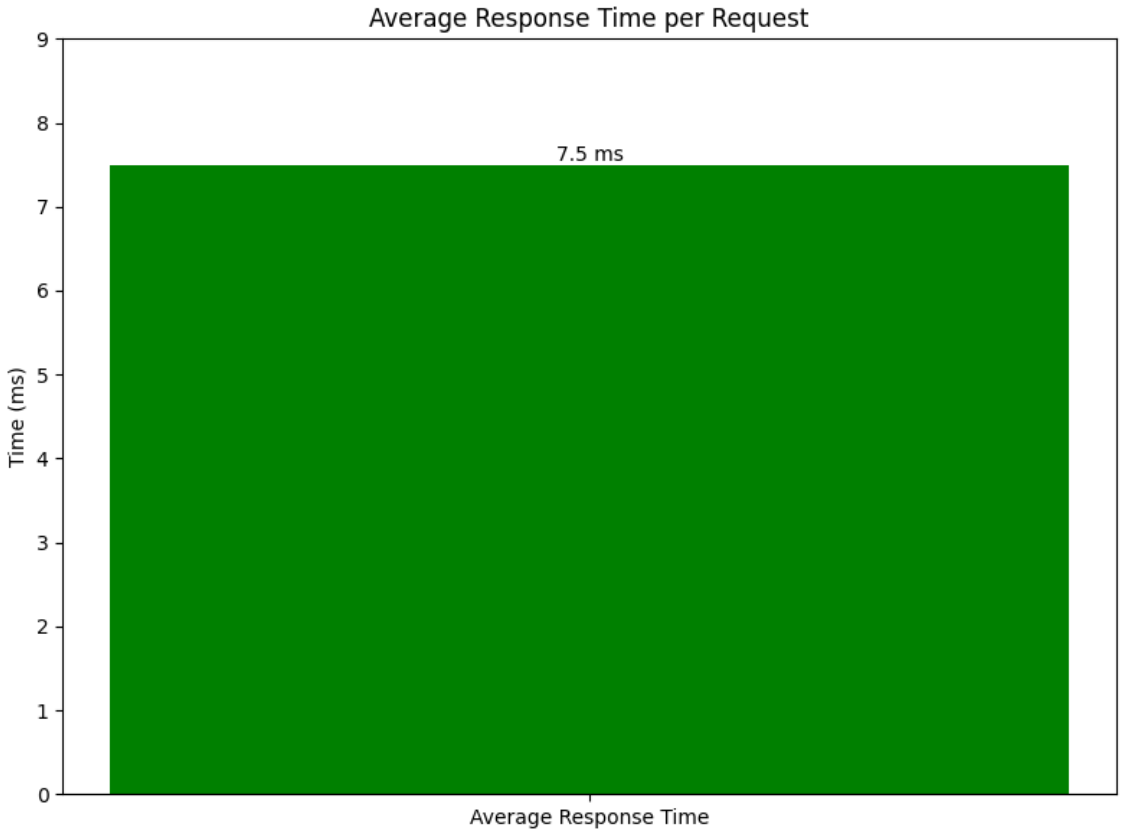
nginx/nginx.conf:

```
upstream backend {
  server app1:5000;
  server app2:5000;
}

server {
  listen 80;
  server_name localhost;

  location / {
    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location /solve {
    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

Milestone 04: Synthetic Workload Generation

Objective: Use httperf to generate synthetic workload to the Nginx container and profile CPU usage. Generate two graphs: average response time and CPU utilization.





Source Code:

cpu_profiler.py:

```
#!/usr/bin/env python3
"""
CPU Profiler for Milestone 04
Monitors CPU usage and saves data to csv file for graphing
"""
import psutil
import time
import csv
import sys
from datetime import datetime

def monitor_cpu(duration=60, interval=1, output_file='cpu_usage.csv'):
    """
    Monitor CPU usage for specified duration

    Args:
        duration: total monitoring time in seconds
        interval: sampling interval in seconds
        output_file: CSV file to save results
    """
    print(f"Starting CPU monitoring for {duration} seconds...")
    print(f"Sampling every {interval} seconds")
    print(f"Data will be saved to: {output_file}")
    print("Press Ctrl+C to stop early\n")

    # Write header
    with open(output_file, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['timestamp', 'cpu_percent', 'cpu_count'])

    start_time = time.time()
    sample_count = 0

    try:
        while (time.time() - start_time) < duration:
            # Get CPU usage
            cpu_percent = psutil.cpu_percent(interval=interval)
            timestamp = datetime.now().isoformat()

            # Write to CSV
```

```
        with open(output_file, 'a', newline='') as f:
            writer = csv.writer(f)
            writer.writerow([timestamp, cpu_percent, psutil.cpu_count()])

        sample_count += 1
        elapsed = time.time() - start_time

        # Print progress
        print(f"[{elapsed:5.1f}s] CPU: {cpu_percent:5.1f}% (Sample {sample_count})",
end='\r')

    except KeyboardInterrupt:
        print(f"\n\nMonitoring stopped by user after {sample_count} samples")

    print(f"\nMonitoring complete. {sample_count} samples saved to {output_file}")

    # Calculate statistics
    try:
        import pandas as pd
        df = pd.read_csv(output_file)
        print(f"\nStatistics:")
        print(f"  Average CPU: {df['cpu_percent'].mean():.2f}%")
        print(f"  Max CPU: {df['cpu_percent'].max():.2f}%")
        print(f"  Min CPU: {df['cpu_percent'].min():.2f}%")
    except ImportError:
        # Fallback if pandas not available
        with open(output_file, 'r') as f:
            reader = csv.reader(f)
            next(reader)  # skip header
            cpu_values = [float(row[1]) for row in reader]
        if cpu_values:
            print(f"\nStatistics:")
            print(f"  Average CPU: {sum(cpu_values)/len(cpu_values):.2f}%")
            print(f"  Max CPU: {max(cpu_values):.2f}%")
            print(f"  Min CPU: {min(cpu_values):.2f}%")

if __name__ == "__main__":
    # Default: monitor for 60 seconds, 1-second intervals
    duration = 60
    if len(sys.argv) > 1:
        try:
            duration = int(sys.argv[1])
        except ValueError:
```

```
        print("Usage: python cpu_profiler.py [duration_seconds]")
        sys.exit(1)

    monitor_cpu(duration=duration)
```

generate_graphs.py:

```
import matplotlib.pyplot as plt
import pandas as pd
import re
import os

def create_cpu_graph(csv_file, output_png):
    # Read the CSV file
    df = pd.read_csv(csv_file)
    # Convert timestamp to datetime for better plotting
    df['timestamp'] = pd.to_datetime(df['timestamp'])

    plt.figure(figsize=(10, 6))
    plt.plot(df['timestamp'], df['cpu_percent'], marker='o', linestyle='-', color='b')
    plt.title('CPU Utilization During Load Test')
    plt.xlabel('Time')
    plt.ylabel('CPU Usage (%)')
    plt.grid(True)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(output_png)
    plt.close()
    print(f"CPU graph saved to {output_png}")

def create_response_time_graph(httperf_file, output_png):
    # Read the httperf output file
    with open(httperf_file, 'r') as f:
        content = f.read()

    # Extract the average response time from the line like:
    # "Reply time [ms]: response 21.4 transfer 0.0"
    match = re.search(r'Reply time \[ms\]: response ([\d.]+)', content)
    if match:
        avg_response_time = float(match.group(1))
    else:
        # Fallback: look for Connection time [ms]: avg
        match = re.search(r'Connection time \[ms\]: avg ([\d.]+)', content)
        if match:
```

```
        avg_response_time = float(match.group(1))
    else:
        # If still not found, use a default or raise an error
        print("Warning: Could not extract response time from httperf output. Using 0.")
        avg_response_time = 0.0

    # Create a bar graph
    plt.figure(figsize=(8, 6))
    plt.bar(['Average Response Time'], [avg_response_time], color='g')
    plt.title('Average Response Time per Request')
    plt.ylabel('Time (ms)')
    plt.ylim(0, avg_response_time * 1.2 if avg_response_time > 0 else 10)  # Add some
headroom
    # Add value on top of the bar
    plt.text(0, avg_response_time, f'{avg_response_time:.1f} ms', ha='center', va='bottom')
    plt.tight_layout()
    plt.savefig(output_png)
    plt.close()
    print(f"Response time graph saved to {output_png}")

if __name__ == "__main__":
    # Define file paths
    cpu_csv = "cpu_usage.csv"
    httperf_txt = "httperf_output.txt"
    graphs_dir = "graphs"

    # Ensure graphs directory exists
    os.makedirs(graphs_dir, exist_ok=True)

    # Create CPU graph
    cpu_graph_path = os.path.join(graphs_dir, "cpu_utilization_graph.png")
    create_cpu_graph(cpu_csv, cpu_graph_path)

    # Create response time graph
    response_time_graph_path = os.path.join(graphs_dir, "response_time_graph.png")
    create_response_time_graph(httperf_txt, response_time_graph_path)

    print("Graph generation complete.")
```